Vite Under the Hood

A conceptual walkthrough of the `class/js/74/usingVite` lesson: what each file and command does, what Vite is doing behind the scenes, and what it's all for.

The outputs below aren't hypothetical. They come from installing, running and building a copy of the lesson project with Vite 8.3.6 on 10 November 2026.

The big picture in one page

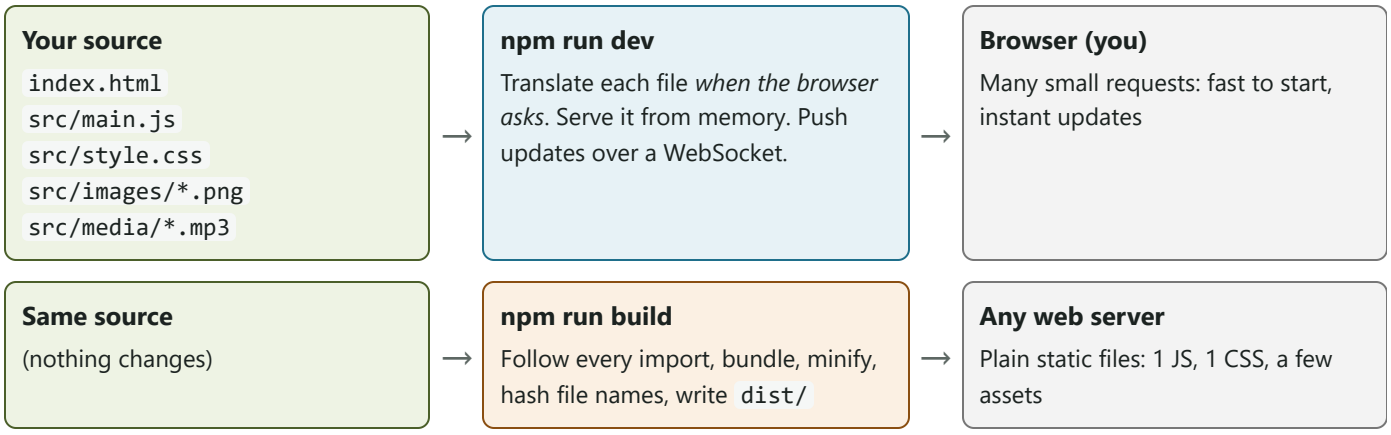
The snake game exists twice in the class folder. `74/snake.js` + `snake.html` is a plain web page: the browser loads the files exactly as they sit on disk. `74/usingVite/` is the *same game*, reorganized so that a tool, Vite, sits between your source files and the browser. The game logic is almost the same. What changed is **how the files reach the browser**.

CORE MENTAL MODEL: THE WORKSHOP AND THE SHIPPING DOCK

Vite has two jobs, and keeping them separate makes everything else easier to follow.

- DEV **The workshop** (`npm run dev`). While you're building, Vite acts as a live *translator at the door*. The browser asks for a file, and Vite translates it into something the browser understands at that moment, one file at a time, with no packing. When you change a file, it tells the browser right away. The goal is **speed of feedback**.
- BUILD **The shipping dock** (`npm run build`). When you're done, Vite packs everything for delivery: it follows every dependency, packs the code into as few boxes as possible, shrinks them, and stamps each box with a fingerprint of its contents. The result goes into `dist/`. The goal is **speed and reliability for the user**.

Your source code in `src/` is the same in both cases. Only the delivery method changes.



The single idea that drives both modes: **everything is a module, and `import` statements are a map of what your page needs**. Once a CSS file, an image or a sound is `import`ed, Vite knows exactly which files the app depends on, so it can translate them in dev and pack only those in build.

1. The problem: what the browser can't do by itself

> **MENTAL MODEL: THE BROWSER IS A STRICT READER**
> A browser only understands HTML, CSS and JavaScript, and only fetches things by **URL**. It doesn't know about your folders, your project, or anything you installed with npm. Whatever path you write is taken literally, relative to the page.

Look at how the non-Vite class version wires things up:

74/snake.html: assets declared in HTML

```
<style> ... </style>
<audio id="crash" src="media/crash.mp3"></audio>
<audio id="crunch" src="media/crunch.mp3"></audio>
<script src="snake.js" type="module"></script>
```

74/snake.js: assets as hard-coded strings

```
snakeHead.src = 'images/snakeHead.png';
appleImg.src = 'images/apple.png';
const crash = document.querySelector('#crash');
```

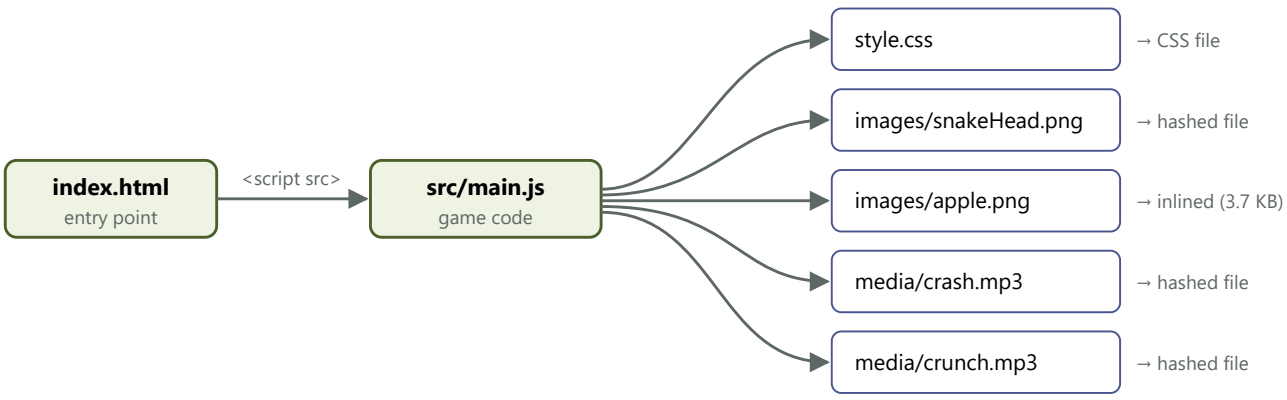
That works, but it has hidden weaknesses that grow with the project:

- **Nothing checks your paths.** A typo in `'images/snakeHed.png'` only shows up at runtime as a broken image. No tool knows that the string is meant to be a file.
- **Caching is unreliable.** When you update `snake.js`, a visitor's browser may keep the old cached copy, because the URL didn't change.
- **No optimization.** Files ship exactly as written: comments, whitespace, long variable names, unused files and all.
- **JS can only import JS.** If you write `import './style.css'` in a plain browser module, the browser refuses: it expected JavaScript and the server returned CSS. The same goes for `import x from './apple.png'`.
- **npm packages don't load.** `import confetti from 'canvas-confetti'` (a "bare" import with no path) means nothing to a browser. Something has to translate it into a real URL inside `node_modules`.

Vite solves all five by putting itself *between* your files and the browser.

2. The key concept: the module graph

Starting from `index.html`, Vite reads every `<script>` and `import` and builds a **dependency graph**: a map of which file needs which. Here is the graph for this lesson:



MENTAL MODEL: IMPORTS ARE A PACKING LIST

In the old version, `main.js` *mentioned* its images in strings, and the HTML *mentioned* the sounds. Nobody had a full list. Now every dependency is `import`ed, so `main.js` is effectively a packing list: "this code needs these five files." Vite reads the list. Anything not on it isn't shipped (except `public/`, covered below). Anything on it that's missing becomes an error you see immediately, not a broken image later.

3. The lesson, step by step

1 Scaffolding the project

The folder contains files the teacher didn't write by hand: a long `.gitignore`, `public/favicon.svg`, `public/icons.svg` (social-media icons the game never uses), and `"version": "0.0.0"`. These are the fingerprints of Vite's starter generator, almost certainly run as:

```
npm create vite@latest usingVite     # then choose: Vanilla → JavaScript
cd usingVite
npm install
```

The generator creates a minimal working app. The teacher then replaced the demo code with the snake game.

> **Where this fits:** scaffolding gives you the three things Vite needs: an **entry** (`index.html` at the project root), a **source folder** (`src/`), and a **recipe** (`package.json`) that names Vite as a tool and defines the commands.

2 `package.json`, `package-lock.json`, `node_modules`

```
{
  "name": "usingvite",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "devDependencies": {
    "vite": "^8.3.6"
  }
}
```

> **MENTAL MODEL: RECIPE, RECEIPT AND PANTRY**
>
> `package.json` is the **recipe**: "this project needs Vite, version 8.3.6 or any newer 8.x" (that's what the caret `^` means). `package-lock.json` is the **receipt**: the *exact* versions that were actually installed, with integrity hashes, so everyone who installs gets byte-for-byte the same tools. `node_modules/` is the **pantry**: the downloaded packages themselves. It's large and can always be rebuilt from the receipt, so `.gitignore` excludes it, which is why it's not in the class folder.

FIELD	WHAT IT MEANS HERE
`"type": "module"`	Tells Node.js to treat `.js` files in this project as ES modules (`import` / `export`), not old-style CommonJS (`require`). This matters for any config files Node runs, such as a future `vite.config.js` .
`"scripts"`	Shortcuts. `npm run dev` runs the `vite` command from `node_modules/.bin` . You don't need Vite installed globally, because each project carries its own exact version.

`"devDependencies"`	Tools needed to *build* the site, not code that runs in the visitor's browser. None of Vite itself ends up in `dist/`.
`"private": true`	A safety catch: it stops `npm publish` from accidentally uploading this app to the public npm registry.

What `npm install` actually downloads (15 packages from the lockfile). These names show what Vite is built from:

- `vite`: the orchestrator. It runs the dev server, plugins and config.
- `rolldown`: the **bundler** used by `vite build`. It follows the module graph and packs it into output files. (Vite 8 replaced its older Rollup/esbuild pair with Rolldown.)
- `lightningcss` and `postcss`: CSS processing and minification.
- `@oxc-project/types`: part of the fast JavaScript parser/transformer toolchain behind Rolldown.
- `fdir`, `tinyglobby`, `picomatch`: fast file finding and matching. `nanoid`, `picocolors`, `source-map-js` and `detect-libc` are small helpers.
- `@rolldown/binding-…` and `lightningcss-win32-x64-msvc` etc.: native, pre-compiled machine code for each operating system. Only the one for *your* system is installed. This is a big reason modern tools are fast: the heavy work runs in compiled Rust, not JavaScript.

3 `index.html` moves to the root and becomes the entry point

```
<!doctype html>
<html lang="en">
  <head>
    ...
    <!--link rel="stylesheet" href="src/style.css" /-->
  </head>
  <body>
    <!--audio id="crash" src="media/crash.mp3"></audio> ... -->
    <canvas id="theCanvas"></canvas>
    <script src="src/main.js" type="module"></script>
  </body>
</html>
```

In many older tools, HTML was an afterthought generated by the bundler. Vite takes the opposite view: **`index.html` is the front door** of the module graph. Vite starts there, finds `<script type="module" src="src/main.js">`, and follows imports outward from that point.

The two commented-out blocks are the lesson's "before" state. The CSS `<link>` and the `<audio>` tags were moved *out* of HTML and *into* JavaScript imports (steps 4 and 6). `type="module"` is required, because Vite relies on the browser's native module system in dev.

4 Importing CSS from JavaScript

```
import './style.css';
```

This line has no variable and doesn't export anything. It's an import purely for its *side effect*: "this module needs these styles applied." A browser can't do this natively. Here's what Vite turns it into:

DEV what the browser receives for `/src/style.css`

```
import { updateStyle, removeStyle }
  from "/@vite/client";
const __vite__css = "body {\r\n  margin: 0; ...";
__vite__updateStyle(__vite__id, __vite__css)
import.meta.hot.accept()
```

The CSS file is **served as JavaScript** that puts a `<style>` tag into the page. Because it "accepts" hot updates, editing the CSS swaps the styles in place without reloading the game.

BUILD `dist/assets/index-CVWbQ70d.css`

```
body{width:100%;height:100%;margin:0}#theCanvas{box-sizing:border-box;border:1px solid #000;display:block}
```

In production it's extracted back out into a real `.css` file, minified (`black` → `#000`, whitespace removed), and linked from the built `index.html`. Same source, two different delivery methods.

5 Importing images: the import's value is a URL

Before (74/snake.js)

```
snakeHead.src = 'images/snakeHead.png';
appleImg.src = 'images/apple.png';
```

After (usingVite/src/main.js)

```
import snakeSrc from './images/snakeHead.png';
import appleSrc from './images/apple.png';
...
snakeHead.src = snakeSrc;
appleImg.src = appleSrc;
```

> **MENTAL MODEL: IMPORTING AN ASSET GIVES YOU A CLAIM TICKET**
> You don't get the image's pixels. You get a **string**: the URL where the image *will* be served. Vite decides what that URL is, and you just use it. That's why the code assigns it to `.src`, exactly as it did with the hard-coded string before.

The ticket's value is different in each mode, which is the whole point:

IMPORT	DEV VALUE	BUILD VALUE
`snakeSrc` (5,338 bytes)	`"/src/images/snakeHead.png"`	`"/assets/snakeHead-B11kA9RX.png"`: copied to `dist/` with a content hash in the name
`appleSrc` (3,783 bytes)	`"/src/images/apple.png"`	`"…"`: **inlined**. The image is embedded inside the JS, and no separate file exists.

Why was the apple inlined but not the head? Vite inlines any asset smaller than 4 KB (`build.assetsInlineLimit`, default 4,096 bytes). For a tiny file, the cost of an extra network request outweighs the cost of storing it as text in the JavaScript, which is about 33% larger. The apple is 3,783 bytes, just under the limit, while the head is 5,338 bytes. You don't change any code for this to happen: because you used `import`, Vite is free to choose the best delivery method.

In dev, Vite answers the request `/src/images/apple.png?import` with a one-line module:

```
export default "/src/images/apple.png"
```

That's literally all an asset import is: a tiny generated module whose default export is a URL.

6 Moving the sounds from HTML into JavaScript

```
// after class commented out audio elements in html, creating and setting src here - from modules
import crashSoundSrc from './media/crash.mp3';
import crunchSoundSrc from './media/crunch.mp3';
const crash = document.createElement('audio');
crash.src = crashSoundSrc;
const crunch = document.createElement('audio');
crunch.src = crunchSoundSrc;
```

This is the same claim-ticket idea as step 5, applied to audio. The old HTML said `src="media/crash.mp3"`. After moving files into `src/`, that path pointed to a folder that no longer exists. Importing from JS fixes the path, makes the dependency visible in the graph, and gets the file hashed in `dist/` (`crash-2TkI9ZAM.mp3`, `crunch-BP5k_2o9.mp3`). Both sounds are over 4 KB, so they stay separate files.

> **Good to know:** Vite *also* processes asset URLs written directly in HTML tags, as long as the path is correct. As a test, adding `<audio src="/src/media/crash.mp3">` to `index.html` built to `<audio src="/assets/crash-2TkI9ZAM.mp3">`. So keeping the `<audio>` tags would have worked with corrected paths. Importing them in JS is a style choice: it keeps every asset the game needs listed at the top of `main.js`.

7 `npm run dev`: the workshop

```
VITE v8.3.6  ready in 149 ms
➜  Local:   http://localhost:5173/
```

"Ready in 149 ms" is surprisingly fast, because **nothing has been processed yet**. The dev server is lazy: it starts listening and waits. Here's the request-by-request story when you open the page:

1. **Browser requests `/`.** Vite sends `index.html` with one line injected at the top of `<head>`:

   ```
   <script type="module" src="/@vite/client"></script>
   ```

   That's the Vite client. It opens a **WebSocket** back to the server so the server can push "file changed" messages to the page.
2. **Browser requests `/src/main.js`.** Vite reads the file from disk, rewrites only the import lines, and leaves the rest untouched:

   ```
   import "/src/style.css";
   import snakeSrc from "/src/images/snakeHead.png?import";
   import appleSrc from "/src/images/apple.png?import";
   import crashSoundSrc from "/src/media/crash.mp3?import";
   ...
   const theCanvas = document.querySelector('#theCanvas');   ← your code, unchanged
   ```

   Relative paths become absolute URLs, and `?import` marks "the JS wants this as a module, not the raw file." (A bare npm import like `'canvas-confetti'` would be rewritten here to a real URL under `/node_modules/.vite/deps/`.)
3. **The browser's own module loader** sees those imports and requests each one. Vite translates each one the same way: CSS → style-injecting JS (step 4), images/audio → `export default "url"` (step 5).
4. **You edit and save a file.** Vite's file watcher notices, works out which modules are affected using the graph, and sends a message over the WebSocket:
   - Edit `style.css` → that module *accepts* hot updates, so new styles are swapped in and **the game keeps running**. This is **Hot Module Replacement (HMR)**.
   - Edit `main.js` → it has no `import.meta.hot.accept()`, so there's no safe way to swap it while running and Vite does a **full page reload**. (Frameworks like React and Vue add accept handlers automatically, which is how component edits keep their state.)

> **MENTAL MODEL: THE TRANSLATOR AT THE DOOR**
> The dev server doesn't build your app ahead of time. It stands between disk and browser and translates **each file the moment it's requested**, and only files that are actually requested. Startup time therefore doesn't grow with project size, and a change only re-translates the file that changed. The responses are marked `Cache-Control: no-cache` so the browser always checks back for fresh code. That's ideal for development but would be too slow for real users, which is why there's a separate build step.

Common mistake: opening `usingVite/index.html` by double-clicking it, or through XAMPP at `http://localhost/Class/class/js/74/usingVite/`, does **not** work. Apache serves `main.js` raw, and the browser fails at line 1 (`import './style.css'`) because it can't import CSS. The source only works *through* Vite's translator (`npm run dev`) or *after* building (`dist/`).

8 `npm run build`: the shipping dock

```
vite v8.3.6 building client environment for production...
✓ 9 modules transformed.
dist/index.html                      0.59 kB │ gzip: 0.34 kB
dist/assets/snakeHead-B11kA9RX.png   5.33 kB
dist/assets/crunch-BP5k_2o9.mp3     16.38 kB
dist/assets/crash-2TkI9ZAM.mp3      23.35 kB
dist/assets/index-CVWbQ70d.css       0.10 kB │ gzip: 0.11 kB
dist/assets/index-Bu5pg987.js        8.01 kB │ gzip: 5.43 kB
✓ built in 67ms
```

The build runs the whole pipeline in one pass. In order:

STAGE	WHAT HAPPENS TO THIS PROJECT
1. Graph	Start at `index.html` and follow every import. "9 modules transformed" = `index.html`, `main.js`, `style.css`, 4 assets, plus small helper modules Vite adds itself (e.g. the module-preload polyfill you can see at the very start of the built JS).
2. Transform	Each file goes through the same plugins as in dev: CSS processed, assets turned into URL strings or inlined.
3. Bundle	**Rolldown** merges all JS modules into one file. The `import` lines disappear because everything now lives in one scope. With a bigger app, code no page uses (*dead code*) is dropped here. This is *tree-shaking*.

4. Minify	Whitespace and comments are removed and local names are shortened:
	`var o=document.querySelector(`#theCanvas`),s=o.getContext(`2d`),c=64;`
	`function l(){o.width=window.innerWidth-window.innerWidth%c,...}`
	`theCanvas` → `o`, `SNAKE_SIZE` → `c`, `sizeCanvas` → `l`, `false` → `!1`. The behaviour is identical but the file is smaller. Gzip compresses it further for transfer (8.01 → 5.43 kB).

5. Hash	Every output file name gets a fingerprint from its contents, such as `index-Bu5pg987.js` (explained below).

6. Rewrite HTML	`index.html` is updated to point at the final files:
	`<script type="module" crossorigin src="/assets/index-Bu5pg987.js"></script>`
	`<link rel="stylesheet" crossorigin href="/assets/index-CVWbQ70d.css">`
	The original `<script src="src/main.js">` is gone. The Vite client is gone too: `dist/` has no dev server and no HMR, just plain static files.

7. Copy `public/`	`favicon.svg` and `icons.svg` are copied to `dist/` *as-is*, with no hashing and no processing.

MENTAL MODEL: CONTENT HASHING IS A FINGERPRINT ON EVERY BOX

`Bu5pg987` is computed from the file's *contents*. Change one character in `main.js`, rebuild, and the JS file gets a new name, and `index.html` is updated to point to it. Leave a file alone and its name stays the same. This makes caching easy: the server can tell browsers to keep `/assets/*` **forever**. An unchanged file is never downloaded twice, and a changed file has a new name, so a stale copy can't be used by mistake. Only the small `index.html` must always be fetched fresh. This is **cache busting**, and it fixes the "visitors see the old version" problem from section 1.

9 `npm run preview`: inspecting the boxes before they ship

`vite preview` starts a simple local static server pointed at `dist/`. It does no translating and has no HMR, so you're testing exactly what users will get. Use it to catch build-only problems: a missing asset, a wrong path, something that behaves differently once minified. It's a *check*, not a production server. In real use you upload `dist/` to any static host (or Apache).

4. `src/` vs. `public/`: two doors for assets

	`SRC/` (IMPORTED)	`PUBLIC/` (COPIED VERBATIM)
How you reference it	`import url from './images/apple.png'`	Absolute path by name: `/favicon.svg`
In the graph?	Yes. Missing files cause a build error.	No. Vite doesn't know whether you use it.
In `dist/`	Hashed name, or inlined if under 4 KB. Only shipped if imported.	Same name, always shipped.
Use for	Almost everything: images, sounds, fonts your code uses	Files that need a fixed, known URL: `favicon.ico`, `robots.txt`, a downloadable PDF

This lesson shows the difference: `public/icons.svg` (social-media icons left over from the starter template) is referenced by nothing, yet it's still copied into `dist/`, along with a 9.5 KB favicon the HTML never links. The game assets in `src/` ship only because `main.js` imports them.

5. Dev vs. build side by side

	NPM RUN DEV	NPM RUN BUILD
When it does work	Lazily, per request	All at once, up front
JS delivery	One file per module, native browser imports	Bundled into one file
Your code	Readable, only import paths rewritten	Minified, names shortened
CSS	JS that injects a `<style>` tag (hot-swappable)	A real minified `.css` file
Asset URLs	`/src/images/snakeHead.png`	`/assets/snakeHead-B11kA9RX.png` or a `data:` URL

Caching	`no-cache`: always fresh	Hashed names: cache forever
Extra code	`/@vite/client` + WebSocket for HMR	None
Where it lives	In memory; nothing written to disk	`dist/` folder (git-ignored)
Optimized for	You: instant start, instant feedback	Users: few, small, cacheable files

6. Practical gotchas for this setup

Deploying `dist/` under XAMPP: absolute paths. By default Vite assumes the site lives at the *root* of a domain, so it writes `src="/assets/index-Bu5pg987.js"`. If you open `http://localhost/Class/class/js/74/usingVite/dist/`, the browser looks for `http://localhost/assets/...`, which doesn't exist, and you get a blank page. **Fix:** tell Vite the base path. The simplest option is a relative base, either per command:

```
npx vite build --base=./
```

or permanently in a `vite.config.js` next to `package.json`:

```
import { defineConfig } from 'vite';
export default defineConfig({ base: './' });
```

Tested on this project: the output became `src="./assets/index-BK6xvzhy.js"`, and the JS references images with `new URL('snakeHead-B11kA9RX.png', import.meta.url)`, which works from any folder.

Running the lesson from a fresh copy. `node_modules/` isn't included (it's git-ignored), so run `npm install` once in `usingVite/`, then `npm run dev`. This Vite version requires Node.js 20.19+ or 22.12+. Your machine has Node 24.18, which is fine.

Paths in imports are relative to the importing file. `'./images/apple.png'` in `src/main.js` means `src/images/apple.png`. In the old version, `'images/apple.png'` was relative to the *HTML page*. Imports are resolved by the file system, but runtime strings are resolved by the page URL. That's one reason imports are safer.

Only `import`ed assets are tracked. If you build a path at runtime, for example `img.src = `./images/${name}.png``, Vite can't see it, so it won't be copied to `dist/`. Either import each file, put it in `public/`, or use `new URL(`./images/${name}.png`, import.meta.url)`, a pattern Vite can recognize.

7. Recap: the whole lesson in one chain

1. **Scaffold** a project → you get an entry (`index.html`), a source folder (`src/`), and a recipe (`package.json`).
2. **Install** → the lockfile gives an exact, repeatable toolset in `node_modules/`: Vite plus Rolldown, Lightning CSS and native binaries.
3. **Turn every dependency into an `import`**: CSS, images, sounds. Your JS becomes the packing list, and each import gives you a claim-ticket URL.
4. **Develop with `npm run dev`** → a translator serves each file on request, and HMR pushes changes instantly.

5. **Ship with `npm run build`** → follow the graph, bundle, minify, inline small assets, fingerprint file names, rewrite the HTML, and write plain static files to `dist/`.
6. **Check with `npm run preview`**, then upload `dist/` anywhere (set `base` if it's not at the domain root).

The game code itself barely changed. The lesson is about *declaring* what your code needs instead of hard-coding where it lives, and letting a tool decide the best way to deliver it in each situation.